

Parallelized Boid Simulation

Josiah Miggiani, Ryan Huang

URL: <https://josiemigg.github.io/parallel-project.html>

SUMMARY:

Boids, short for “bird-oids,” are independent actors which can collectively simulate the complex aggregate motion of a flock of birds, a school of fish, or even crowd movement of humans. We will create an optimized implementation of this particle simulation and compare performance characteristics with a sequential implementation. We also intend to develop a simple visualizer.

BACKGROUND:

Boids were first proposed by Craig W. Reynolds in his paper submission to SIGGRAPH '87, “Flocks, Herds, and Schools: A Distributed Behavioral Model.” In his words, “The aggregate motion of the simulated flock is the result of the *dense interaction* of the *relatively simple behaviors* of the individual simulated birds.” Though each boid is independently responsible for deciding the course of motion it takes, these decisions are informed by each boid’s local perception of their dynamic environment. The interplay between behavioral rules programmed into the boids and their understanding of their local surroundings (such as nearby boids) is what produces such compelling aggregate dynamics.

In order to better understand how boid simulation can be parallelized, it’s first necessary to understand the basic algorithm that dictates boid flight planning. All boid behavior is constrained by three fundamental rules:

1. Alignment (or velocity matching)
2. Separation (or collision avoidance)
3. Coherence (or flock centering)

Boid alignment seeks to maintain a similar velocity to a boid’s nearby flockmates. Velocity is a vector quantity, representing heading (orientation) and speed. Separation requires that each boid attempts to avoid flying into other boids. As a boid nears in spatial proximity to another, it will alter its heading to avoid collision. Lastly, coherence dictates that boids seek to fly closer towards nearby boids. More specifically, it makes a boid want to be near the center of the flock. Coherence causes the boid to take a heading that moves it closer to the centroid of nearby flockmates.

Here’s what some general pseudocode might look like of the algorithm.

```
Python
# coefficients we can define at runtime
COHERENCE = x
SEPARATION = y
```

```
ALIGNMENT = z
```

```
def boid():  
    def centerFlock():  
        localFlock = self.neighbors()  
        centerPos = average position of localFlock  
        adjust self.vel to head towards centerPos, regulated by COHERENCE  
  
    def avoidCollision():  
        localFlock = self.neighbors()  
        for other in localFlock:  
            if distance(self, other) < SEPARATION:  
                point self.vel away from other.pos, regulated by SEPARATION  
  
    def matchVelocity():  
        localFlock = self.neighbors()  
        avgVel = average velocity of localFlock  
        adjust self.vel to approach avgVel, regulated by ALIGNMENT  
  
    def neighbors():  
        localFlock = []  
        for other in boids:  
            if distance(self, other) < self.perception_radius:  
                add other to localFlock  
        return localFlock  
  
    def constrain():  
        clamp self.vel to [minSpeed, maxSpeed]  
        if (self.pos + self.vel) outside spatial bounds:  
            point velocity back towards valid simulation space  
  
    def step():  
        self.centerFlock()  
        self.avoidCollision()  
        self.matchVelocity()  
        # e.g. stay within spatial boundaries, enforce max/min speed  
        self.constrain()  
        self.updatePosition() # based on new velocity  
  
def main():  
    for timestep in simulation:  
        for boid in boids:  
            boid.step()
```

Note that this pseudocode is relatively simple, and does not present a more sophisticated model, such as one that considers a Newtonian approach where boid mass is held constant. Nonetheless, the pseudocode makes it apparent that the problem can strongly benefit from processing boids in parallel. However, it's also clear that there are many dependencies between boids, and each step for each boid cannot be truly independently calculated.

THE CHALLENGE:

Workload

The boids will each take information from its surroundings before making decisions. These boids each must update itself for the next time step. Some characteristics that are specific to this workload:

1. **Communication:** The amount of information that a boid needs depends on the vision parameter given to the boids. Since the boids are constantly taking lots of information from its surroundings regarding other boids, there is a high communication to computation ratio as they constantly gather information from shared data.
2. **Divergent Execution:** Boids can change behavior quickly. The information that a boid gathers might lead to a lot of divergent decisions as boids decide whether to flock to a center or avoid colliding into other boids.

Constraints

There are some challenges with parallelizing the above workload:

1. **Stale Data:** Individual boids require information from their surroundings in order to determine their actions. This shared data will be dynamically updated as the simulation progresses, so we will need to explore creative solutions for efficiently updating shared data to ensure boids do not make decisions on stale data.
2. **Efficiency:** We must also ensure that accessing this data is done efficiently. The data access patterns of these boids can be irregular especially as the shared data and the boids themselves move around. We will have to explore solutions which allow boids to efficiently find critical information while maintaining temporal and spatial locality as best as possible

RESOURCES:

There are numerous resources available online to help us get started. As mentioned earlier, Craig Reynold's paper¹ will serve as the primary source of general understanding of the model. However, the paper does not explicitly present any algorithmic code nor pseudocode, and so we turn to other references to fill in these gaps. Thomas Rouch, in his [Medium article](#) presents his implementation of boids (including visualization via OpenGL) in C++, and publicizes his [GitHub repository](#) as well. He provides a solid base to work from (if heavyweight thanks to 3D assets), though we expect to pare down on the amount of features he provides, such as (dynamic)

obstacles and goals. We will extend this code to be able to simulate without graphics for benchmarking purposes. We expect to run the application locally as well as the GHC clusters.

GOALS AND DELIVERABLES:

What we plan to achieve: We plan to improve upon the original paper's naive algorithm. As originally described in Reynold's paper: "A naive implementation of the basic flocking algorithm would grow in complexity as the order of the square of the flock's population ($O(N^2)$)" ^[1]. We believe we can improve the algorithmic Big-O complexity by some constant factor by leveraging intelligent spatial binning of boids for more efficient local world perception.

We plan to analyze our implementations considering both time-constrained and problem-constrained scaling:

1. Time-constrained: Since this is a simulation, we want to see the simulation update with a reasonable frame rate. We will attempt to push the number of boids we can concurrently simulate within the same time frame demonstrating an improvement to the more realistic time constrained problem.
2. Problem-constrained: We also want to try seeing how our implementation might improve the simulation in a problem constrained manner. We can constrain the number of boids and parameters along with the total number of iterations. Then, comparing the time it takes to finish the simulation between the sequential and parallel approach, we can determine our total speedup.

Plans:

1. Optimize the boid algorithm using binning techniques to minimize the algorithmic complexity of boid querying
2. Use parallel programming techniques to improve the speed of boid behavior computation
3. Develop and present graphs to show how overall speed can be increased within a certain specified number of iterations
4. Present animations of our implementation demonstrating the number of boids that can be displayed at a similar frame rate between our optimized program in comparison to naive program.

What we hope to achieve:

1. Algorithmic Complexity: We hope to achieve an amortized algorithmic complexity of $O(N \log N)$ for N boids. We think this should be possible with sophisticated shared memory usage that can reduce on the number of boids visited when handling separation, alignment and flock centering on each tick.
2. Number of Boids: We hope to increase the limit on the number of boids rendered within a certain time frame by ~6-8x. We think this is possible because our algorithmic improvements will allow our program to reach its cap at a much slower rate. Also the 8 cores will have an ideal speedup of 8x, so this is an appropriate expectation of how many more boids we can render.

3. Overall Speedup: We hope to increase the total speedup of the program by ~10x. Again the 8 cores will have an ideal speedup of 8x, when combined with our improved algorithm we believe that this total speedup is achievable.

PLATFORM CHOICE:

For our project, we will be developing in C++, and parallelizing our implementation using OpenMP. We've decided to use OpenMP over CUDA and OpenMPI for a variety of reasons. Firstly, each boid is an independent agent, meaning execution will quickly diverge between boids. This does not particularly lend itself to a CUDA implementation, where SIMD is the primary source of execution speedup. Though conditional divergence is supported, it reduces overall efficiency. Secondly, each boid maintains some local perception of the world around it, such as flockmates. This is a dynamic grouping which can change from one simulation tick to the next, and so it would be unsustainable for each boid to query the global positions of all other boids, assuming a process was assigned a boid (or some share of boids) each. While there are other potential approaches (perhaps processes divide up simulation space), the amount of communication required would not work well for message passing. OpenMP works well as it supports shared memory, and threads execute independently.

SCHEDULE:

Week 0: (3/24 – 3/30)

- Deploy naive, sequential boid simulation
- Begin graphics-less extension to enable benchmarking without visualization

Week 1: (3/31 – 4/6)

- Begin basic OpenMP implementation of parallelized simulator
- Finish graphics-less extension for benchmarking

Week 2: (4/7 – 4/13)

- Complete basic OpenMP implementation
- Collect initial results for milestone report
- Begin milestone report

Week 3: (4/14 – 4/20)

- Finish milestone report (due Tuesday, 4/15 @ 11:59pm)
- Begin sophisticated binning implementation (dynamic? BVH-like structure?)

Week 4: (4/21 – 4/27)

- Finalize sophisticated shared memory approach
- Evaluate all three approaches
- Begin final report and poster submissions

Week 5: (4/28 – 4/29)

- Finish final report and submit (due Monday, 4/28 @ 11:59pm)
- Present at poster session (Tuesday, 4/29 @ 1 – 4pm or 5:30 – 8:30pm)

REFERENCES:

[1] – Reynolds, C. W. (1987). Flocks, herds, and schools: A distributed behavioral model. Computer Graphics, 21(4), 25-34.